boundHTS: A R package for coherent hierarchical forecasting of bounded outcomes

by Hannah Comiskey, David Frazier, and Ole Maneesoonthorn

Abstract An abstract of less than 150 words.

1 Introduction

A hierarchical time series is a multivariate time series subject to linear aggregation constraints (Hyndman, 2022). Forecasts for such series are considered coherent when lower-level predictions aggregate exactly to higher levels, satisfying the system's aggregation constraints. In practice, base forecasts are often produced independently at each level, which can lead to incoherence. Forecast reconciliation provides a principled post-processing framework that adjusts these base forecasts to produce aggregate-consistent predictions while retaining as much of the original information as possible (Hyndman and Athanasopoulos, 2021). Forecasts may be either point forecasts, representing single most-likely future values, or probabilistic forecasts, representing a distribution over possible future outcomes. A comprehensive review of point and probabilistic reconciliation methods is provided by Athanasopoulos et al. (2024).

Several R packages on CRAN implement forecast reconciliation across diverse scenarios, including point versus probabilistic forecasts, discrete versus continuous data, and different hierarchical structures (e.g., temporal or cross-sectional). While forecast reconciliation has been studied extensively for discrete and continuous-valued time series, comparatively little attention has been given to hierarchical forecasting under bounded constraints.

One of the most widely used packages for point-forecast reconciliation of hierarchical and grouped time series is **hts** (Hyndman et al., 2024). This package implements classical methods such as bottom-up (BU), where bottom-level forecasts are aggregated to form higher-level predictions, and top-down (TD), where parent-level forecasts are disaggregated to child nodes. It also provides more advanced methods, including the Optimal Combination (OC) approach of Hyndman et al. (2011) and the Minimum Trace (MinT) method of Wickramasuriya et al. (2019). OC uses a regression-based framework to adjust forecasts at the bottom level so that they aggregate coherently, while MinT formulates reconciliation as a constrained optimization problem that minimizes the trace of the reconciled forecast error covariance matrix. The **fabletools** package extends hierarchical reconciliation to the tidyverse ecosystem, providing BU, TD, middle-out (MO), and MinT methods (O'Hara-Wild et al., 2025). Both **hts** and **fabletools** focus on point forecasts for cross-sectional hierarchies, assuming underlying Gaussian distributions. For more complex or non-standard structures, including temporal hierarchies, **FoReco** offers flexible reconciliation methods with efficient sparse matrix handling, though it also targets point forecasts only (Girolimetto and Di Fonzo, 2024).

For probabilistic reconciliation, the works of Taieb et al. (2017) and Panagiotelis et al. (2023) have been foundational. The **ProbReco** package explicitly frames reconciliation as an optimization problem based on proper scoring rules, generating coherent probabilistic samples by minimizing the Energy Score (Gneiting and Raftery, 2007; Panagiotelis et al., 2023; Panagiotelis, 2025). However like the aforementioned packages, this method also only considers reconciliation of data generated from Gaussian distributions. Many empirical applications further involve discrete or bounded supports, introducing additional challenges that are not adequately addressed by these standard Gaussian-based reconciliation methods. **bayesRecon** enforces coherence through conditioning under aggregation constraints. It supports Gaussian data as well as count data, employing Bottom-Up Importance Sampling (BUIS) to produce lower-bounded, aggregate-consistent probabilistic forecasts (Corani et al., 2023; Azzimonti et al., 2025). However, this package does not deal with fully-bounded distributions like the Beta distribution for the reconciliation of proportions.

To address these challenges, we present the **boundHTS** package which generates coherent hierarchical forecasts for bounded continuous and lower-bounded discrete variables. Our contribution lies in providing a method to obtain coherent and accurate forecast densities for both bounded and lower-bounded densities. The approach respects the natural bounds of parent distributions. We demonstrate its performance using simulated Beta and Poisson hierarchical data structures.

2 Theory and Methods unpinning boundHTS

To begin, let m be the number of series at the bottom level, and n be the total number of series in the hierarchical set. We denote the bottom series by $\boldsymbol{b}_t$, such that $\boldsymbol{b}_t = [b_{1,t}, b_{2,t}, \dots, b_{m,t}]$ which take values in the probability space $(\mathcal{B}, \mathcal{F}_m, \mu_m)$ where $\mathcal{B} = \prod_{i=1}^{m} \mathcal{Y}_i$. Similarly, let $\boldsymbol{u}_t$ denote the m'-dimensional vector of upper-level series observations. Here, $\boldsymbol{u}_t = [u_{1,t}, u_{2,t}, \dots, u_{m',t}]$, with $m' \leq m$ and $m' + m = n$. The vector $\boldsymbol{u}_t$ take values in the probability space $(\mathcal{A}, \mathcal{F}_u, \mu_u)$. Let $\boldsymbol{y}_t$ denote the vector of series at time t, and $y_{i,t}$ the value at series i and time t, $i = 1, \cdots, n$, and $t \leq T$. Therefore,

$$y_t = (b_t', u_t')' \in \mathcal{Y} := \mathcal{B} \times \mathcal{A}, \tag{1}$$

We assume that the relationship between $\boldsymbol{b}_t$ and $\boldsymbol{u}_t$ can be described using n equality constraints. We define the reconciliation function as $g(\cdot)$ with known parameters $\gamma \in \Gamma$ in $g : \mathcal{Y} \times \Gamma \mapsto \mathcal{Y}$ such that for any given t,

$$g((b_t', u_t')', \gamma) = 0. \tag{2}$$

Let the series in the hierarchical structure be mutually independent. Under this assumption, aggregation via convolution is directly analogous to a bottom-up reconciliation strategy. Let the forecast distribution of bottom-level series m at time t be given by,

$$b_{m,t} \sim f_{m,t}(\cdot).$$

Under a bottom-up aggregation scheme, the forecast density of $u_{m',t}$ may be obtained via convolution where,

$$\begin{aligned} f_{n,t}(u_{m',t}) &= f_1 * f_2 \cdots * f_{m,t}(u_{m',t}), \\ &= \int \cdots \int \prod_{i=1}^{m-1} f_{i,t}(b_{i,t}) f_{m,t}(u_{m',t} - \sum_{i=1}^{m-1} b_{i,t}) db \end{aligned} \tag{3}$$

In practice, (3) may be approximated using Monte Carlo integration making it computationally simple to evaluate.

Tilting is a well-established process that allows for the incorporation of additional information into the convoluted density to change the shape of the density, while preserving its support (Giacomini and Ragusa, 2014; Krüger et al., 2017; West, 2024). It incorporates a set of theoretical restrictions, expressed as k moment conditions, to modify the probability distribution via re-weighting the density or mass function. For example, let $u_{m',t} \sim f_{m',t}(m', t)$ be the base forecast, with the mean of this forecast given by $\hat{u}_{m',t}$. Then, a coherent mean prediction that minimises the bias resulting from the bottom-up convolution of lower series, can be generated by tilting the base density towards a updated density, $f^*_{m',t}(m', t)$, with the first moment matching $\hat{u}_{m',t}$. Therefore,

$$f^*_{n,t}(u_{m',t}) = \frac{\exp(\gamma_{m',t} u_{m',t}) f_{n,t}(u_{m',t})}{\int \exp(\gamma_{m',t} u_{m',t}) f_{n,t}(u_{m',t}) du}, \tag{4}$$

where $\gamma_{m',t}$ is the tilting parameter that minimises the residual between the target mean and the mean of the tilted density for series m' at time t. This tilted density is approximated by numerical integration (Burden and Faires, 2015). When a closed-form expression for the tilted predictive density is available, tilting parameters can be estimated sequentially for each parent node in a hierarchical structure. In such settings, the predictive density at a parent node can be constructed by aggregating the predictive densities of its child nodes, and the corresponding tilting parameter can be obtained via the moment-matching procedure described above. This sequential approach proceeds up the hierarchy, using the aggregated (and possibly tilted) densities at lower levels as the starting point for higher-level nodes. Estimating tilting parameters sequentially, rather than independently across all series, increases the effective degrees of freedom and provides greater flexibility in the resulting predictive densities. When a closed-form expression for the tilted predictive density is unavailable, we recommend tilting each series independently. Specifically, we first use convolution to estimate the predictive density of each aggregated series, and then tilt each estimated density toward its predictive mean.

3 Using boundHTS

The functions of boundHTS

The package provides pre-defined convolution functions for Poisson, Beta, zero-inflated Beta (ZIB), and zero-one inflated Beta (ZOIB) distributions. These functions can operate on either point estimates or Monte Carlo samples of the bottom-level series. They are parallelised to run efficiently across the full range of evaluation inputs.

- **Functions for point estimate inputs**
  - `Poisson_convolution_density_point_parallel`
  - `Beta_convolution_density_point_parallel`
  - `ZIB_convolution_density_point_parallel`
  - `ZOIB_convolution_density_point_parallel`
- **Functions for sample-based inputs**
  - `Poisson_convolution_density_parallel`
  - `Beta_convolution_density_parallel`
  - `ZIB_convolution_density_parallel`
  - `ZOIB_convolution_density_parallel`

Additionally, the package includes the `tilt_density()` function, which exponentially tilts a density toward a specified target mean. This function supports both discrete and continuous distributions, providing a flexible tool for adjusting predictive densities.

Case 1: Aggregating count forecasts

The data

To begin, we consider a two-tier cross-sectional hierarchical time series where the observations are simulated from low-count Poisson distributions. The hierarchical structure is given by Figure @ref(fig:sim_poisson_setup). We consider a scenario where four bottom series (M=4) and 2000 observations (N=2000). The rate parameters of the bottom series are generated using generalised linear models with a log-link. These rate parameters then generate low-value counts via the Poisson distribution. Additional variability is introduced to the bottom series counts using integer values shocks to each series to perturb these series subject to a strict non-negativity constraint. To generate the top-level series, the rate parameters of the bottom series are aggregated. The complete procedure for the data simulation procedure can be found in the Supplementary Materials. This simulated data is stored as `poisson_sim_data` within **boundHTS**. To evaluate the utility of our method, we split the data into 1750 training observations and 250 test observations.

Modelling the base forecasts

To obtain base forecasts, we fit standard autoregressive order(1) models independently to each node in the hierarchy using `tsglm()` from the **tscount** package. These models serve as a simple and reproducible forecasting baseline; our primary contribution lies in the reconciliation methodology applied to the resulting forecasts. For each node, we generate a series of rolling one-step-ahead forecasts. At each step, we record both the in-sample fitted values and the one-step-ahead predictive mean for the rate parameter of each observation. These values are subsequently used to construct a predictive point forecast for each series. These predictions form the base point forecasts that are subsequently reconciled. The implementation is provided for reproducibility, but the reconciliation framework is model-agnostic and can be applied to any suitable base forecasts.

The procedure requires:

- For all nodes in the series,
  - estimates of the rate parameter of the Poisson distribution.
- A discrete support grid over which the PDFs are evaluated

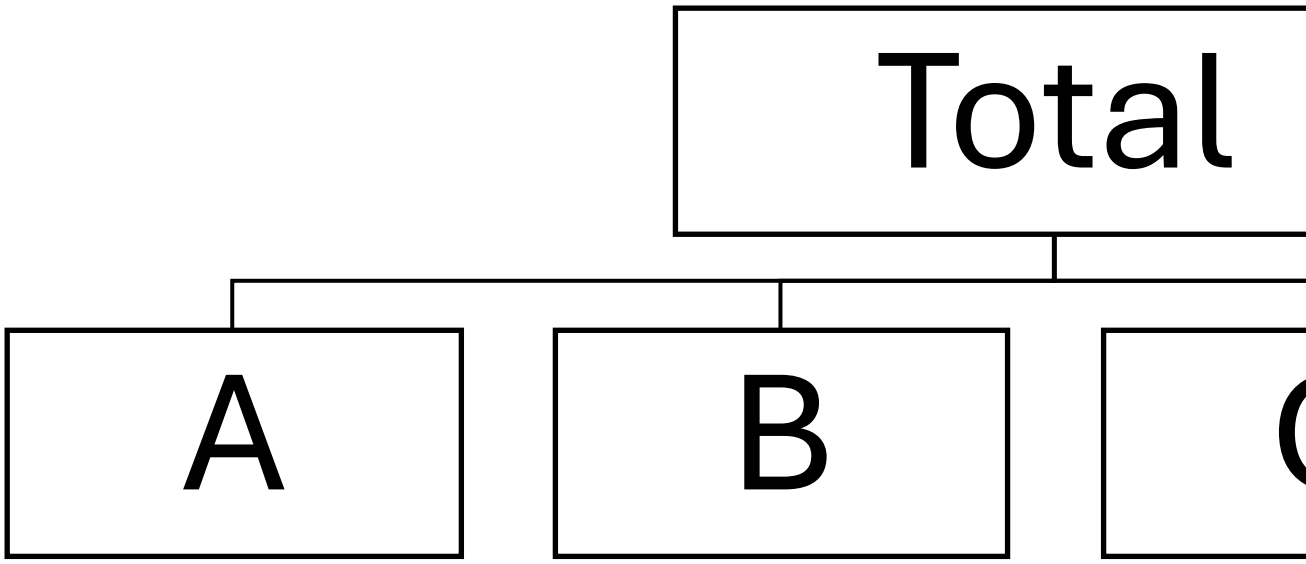

Figure 1: (#fig:sim_poisson_setup)The two-level hierarchical set up of the Poisson simulation. The four child nodes (A, B, C, D) aggregate to the parent node, Total.

Coherent hierarchical forecasting

We construct a coherent probabilistic forecast for the top-level series by combining bottom-level Poisson forecasts through convolution and subsequently enforcing moment consistency via exponential tilting. As the sum of independent Poisson random variables is also Poisson, we may use a stepwise procedure in which lower-level tilted densities are convolved to form the starting density of their parent node. This ensures that the resulting probability mass function (PMF) respects aggregation constraints while aligning with the predictive means obtained from the base models.

```
library(ggplot2)
library(boundHTS)

# Forecast origin
t_idx <- 2000
all_series <- c("Tot","AA", "AB", "BA", "BB")
bottom_series <- c("AA", "AB", "BA", "BB")
z_values <- seq(from = 0, to = c(max(pois_data$Tot)+10))  # support for PMF

# Example: one-step-ahead Poisson predictive means
# Fit GLM to bottom-level series
fit_list <- lapply(all_series, function(series) {
 tscount::tsglm(as.vector(unlist(pois_data[seq_len(t_idx-1), series])), model = list(past_obs = 1), link = "log
})

# Predictive means at time t
lambda_pred <- sapply(fit_list, function(fit) {
   predict(fit, n.ahead = 1)$pred
})
```

The next step is to generate the aggregated series density using convolution. This is accomplished with the `Poisson_convolution_density_point_parallel()` function, which takes the grid of discrete values (`z_values`) and the predictive rate estimates of the bottom-level series (`lambda_vector`). Once the convolution density is computed, it is adjusted, or "tilted", to align with the predictive mean of the top-level series. The `tilt_density()` function performs this adjustment, taking as inputs the predictive mean, the grid of discrete values, and the convoluted density.

```
# Convolution density for total-level series
```

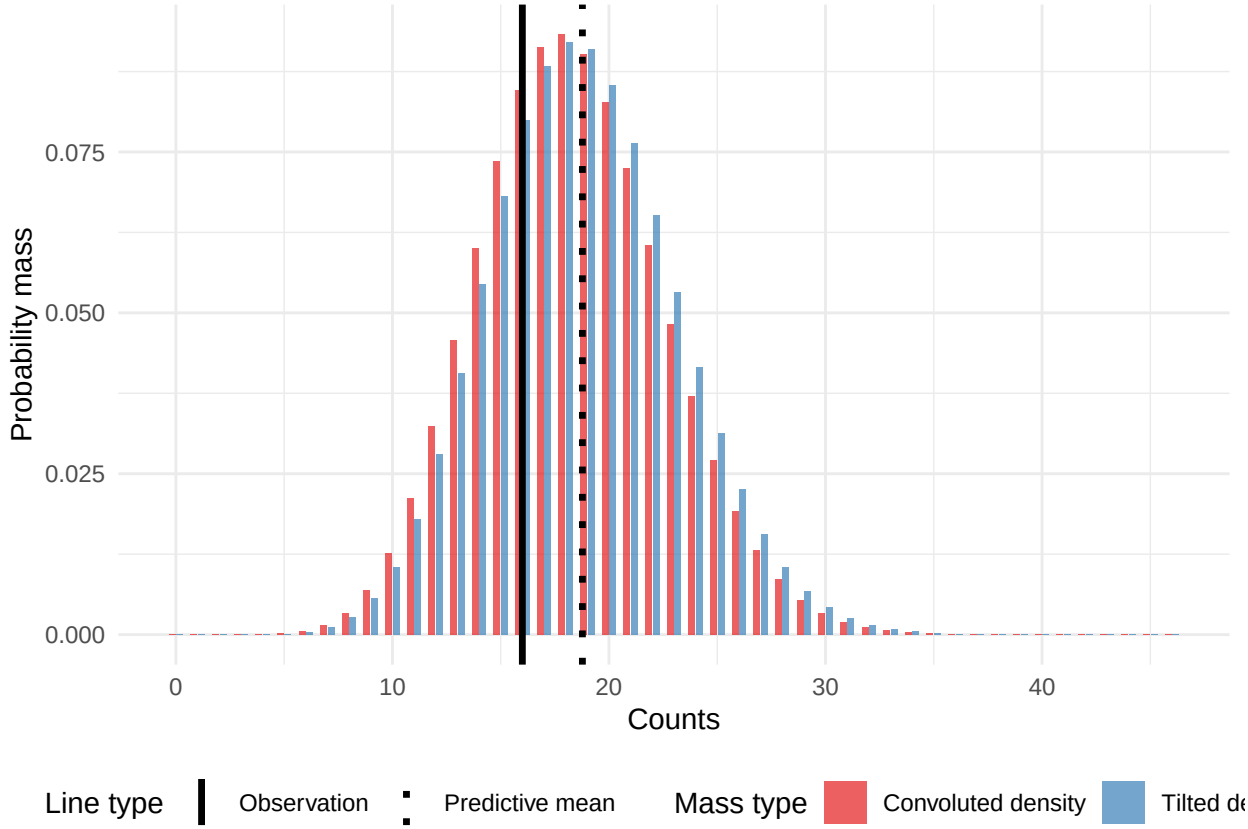

Figure 2: The tilted and convoluted density of the top series using Poisson distributed bottom series for a given time point. The dark red colour captures the density of the convolution, the blue represents the tilted denisty and the dashed black line indicates the predictive mean of the top series.

```
conv_density <- Poisson_convolution_density_point_parallel(z_values = z_values,
                                                           lambda_vector = lambda_pred[-1])

# Construct tilted density to match predictive mean
f_tilted <- tilt_density(mu_theory = lambda_pred[1],
                         y_vals = z_values,
                         f_y = conv_density,
                         discrete = TRUE)
```

Figure 2 displays the exponentially tilted predictive mass functions for the top and bottom level series at a representative forecast origin. The unadjusted Poisson distribution implied by aggregation is shifted via exponential tilting so that its expected value coincides with the predictive mean from the base model. As expected, the adjustment primarily alters the tail behaviour of the distribution while maintaining the aggregation constraint.

Case 2: Aggregating proportional forecasts

The data

To begin, we consider a simulated hierarchical Beta dataset with a two-level structure. The hierarchical structure is given by Figure @ref(fig:sim_beta_setup). Due to the compositional nature of the data, the values across nodes are deterministically linked. As a result, only half of the full hierarchical tree must be explicitly simulated, since the remaining nodes can be obtained through deterministic aggregation. The bottom-level series (AA, AB) are constructed from auto-regressive latent processes on the logit scale with correlated Gaussian noise terms. A full description of this data generating process can be found in the Appendix. In this instance, the number of bottom-series is $M = 2$ and the number of observation is $N = 250$.

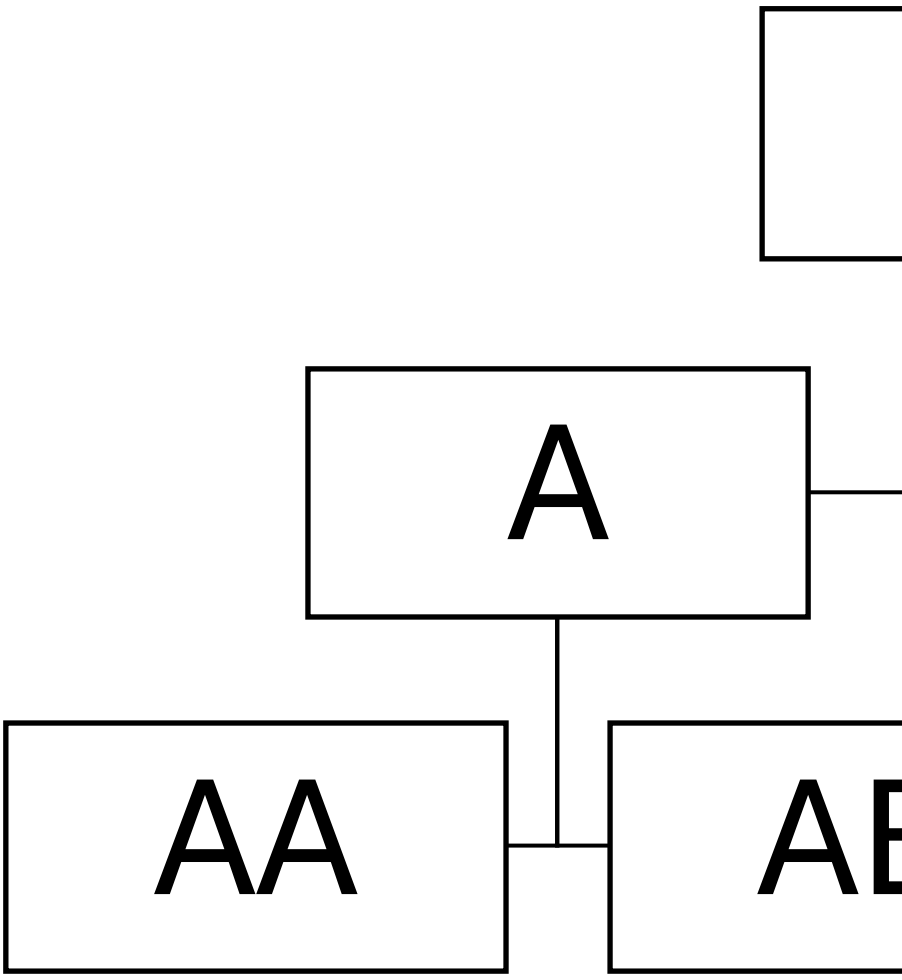

Figure 3: (#fig:sim_beta_setup)The two-level hierarchical set up of the Beta simulation. The two child nodes (AA, AB) aggregate to the parent node, A.

Modelling the base forecasts

To obtain base forecasts, we fit standard linear regression models independently to each logit-transformed node in the hierarchy using `auto.arima()` from the **forecast** package. For each node, we generate a series of rolling one-step-ahead forecasts. At each step, we record both the in-sample fitted values and the one-step-ahead predictive mean for the mean and variance parameters of each observation. These values are subsequently used to construct a predictive point forecast for each series. These predictions form the base point forecasts that are subsequently reconciled.

The procedure requires, - For all nodes in the series: - mean and variance estimates if using the Normal distribution - shape estimates if using the Beta distribution - A continuous fine support grid over which the PDFs are evaluated

Coherent hierarchical forecasting

Like before, we construct a coherent probabilistic forecast for the top-level series by combining bottom-level weighted Beta forecasts through convolution and subsequently enforcing moment consistency via exponential tilting. To begin, we assume the bottom series is given by $b_{m,t} \sim \text{Beta}(\mu_{m,t} * \phi_m, (1 - \mu_{m,t}) * \phi_m)$ and let the mean of these forecasts be given by $\hat{\mu}_{m,t}$ which is bounded by [0,1]. Conditional on $b_{m,t}$, the density of the aggregated top series proportion, $u_{A,t}$, is then approximated via convolutions. A closed form solution for the convolution of Beta distributions does not exist, as such it is necessary to estimate the integral via Monte-Carlo integration. Specifically, the density of $u_{A,t}$ is defined as,

$$f_{A,t}(u_{A,t}) = \int \cdots \int \left(\prod_{i=1}^{m-1} f_{i,t}(b_{i,t}) \right) f_{m,t}(u_{m',t} - \sum_{i=1}^{m-1} b_{i,t})\, db. \tag{5}$$

We begin by fitting independent regression models on the logit transformed proportions of each series and storing each series fit in a list.

```
all_series <- c("A","AA","AB")
bottom_series <- c("AA","AB")
n_bottom <- 2

fit_list <- lapply(all_series, function(s) {
  forecast::auto.arima(beta_sim_data[1:(t_idx-1), paste0("logit_", s)])
})
```

Using the fitted models, we obtain one-step-ahead forecasts on the logit scale. The predictive mean is extracted directly, while the predictive variance is approximated via Monte Carlo simulation. These moments are then back-transformed to the unit interval using the Delta method. Finally, the parameters of the Beta distribution are recovered from the estimated mean and standard deviation using the `beta_params()` function.

```
logit_mu_pred <- sapply(fit_list, predict, h=1)
logit_mu_pred <- as.vector(unlist(logit_mu_pred[1,]))

logit_sim_pred <- lapply(fit_list, function(fit) {
  replicate(5000, simulate(fit, nsim=1, future=TRUE)[[1]])
})

logit_var_pred <- sapply(logit_sim_pred, function(x) var(x))

# Back transform via the Delta method
prop_pred <- 1 / (1 + exp(-logit_mu_pred))
prop_var <- (prop_pred * (1 - prop_pred))^2 * logit_var_pred

# Calculate the beta distribution parameters
top_params <- beta_params(prop_pred[1], sqrt(prop_var[1]))

bottom_params <- list()
for(b in 1:n_bottom) {
  bottom_params[[b]] <- beta_params(prop_pred[b+1], sqrt(prop_var[b+1]))
}
```

The bottom series are rescaled to the interval $[0, w_m]$ using the `rBeta_ab()` from **ExtDist**, where where w_m denotes the aggregation weight for bottom series m. This rescaling facilitates aggregation through convolution. A set of weighted samples is generated for each bottom-level series and stored in a three-dimensional array. The convolution-based density of the aggregated series is then evaluated over a fine grid on [0,1] using the `Beta_convolution_density_point_parallel()` function. This function takes as inputs the evaluation grid, the parameters of the final bottom-level series, the array of weighted samples, and the corresponding aggregation weight. Finally, the resulting density is tilted toward the predictive mean to ensure coherence with the marginal forecast. This is achieved using the `tilt_density()` function.

```
z_values <- seq(0,1,length.out=1000)

weighted_samps <- array(NA, dim=c(100,100,n_bottom))

for(b in seq_along(bottom_series)) {
  weighted_samps[,,b] <- matrix(ExtDist::rBeta_ab(100*100,
                                    shape1 = bottom_params[[b]][1],
                                    shape2 = bottom_params[[b]][2],
                                    a=0,
                                    b=weights_bottom[b]), 100, 100)
}

Density_top <- Beta_convolution_density_point_parallel(z_values = z_values,
                                          alpha_point = bottom_params[[n_bottom]][1],
                                          beta_point  = bottom_params[[n_bottom]][2],
                                                    weighted_samps = weighted_samps,
                                                  weights = weights_bottom[n_bottom]
                                                    )

f_tilted <- tilt_density(mu_theory = prop_pred[1],
                         y_vals = z_values,
                         f_y = Density_top,
                         discrete=FALSE)
```

Figure 4 displays the exponentially tilted probability density functions for the top and bottom level series at a representative forecast origin. The unadjusted convoluted Beta distribution implied by aggregation is shifted via exponential tilting so that its expected value coincides with the predictive mean from the base model.

4 Conclusions

The **boundHTS** is an R package that leverages convolution methods combined with exponential tilting to generate coherent hierarchical forecasts for both discrete and continuous bounded distributions. To the best of our knowledge, it is the first package to address coherent estimation for bounded hierarchical series within a unified framework. The development and dissemination of **boundHTS** as an R package aligns with the findability, accessibility, interoperability, and reusability (FAIR) principles of scientific data (GO FAIR Initiative, 2016). In particular, the package includes well-documented data sources, clearly structured and annotated code, and modular functionality that facilitates reuse and extension by other researchers and practitioners.

5 Appendix

Data generating process for Case 1: Hierarchical Poisson distributions.

The bottom-level series are constructed using a generalized linear model with an auto-regressive term and quadratic log-link. For each series $m = 1, \ldots, M$, regression coefficients are given by,

$$\theta_m \sim \text{Uniform}(0.4,\, 0.5), \quad \nu_{m,t} \sim N(0, 0.1^2)$$

The Poisson rate parameter is then defined as

$$\lambda_{m,t} = \exp\big(\theta_m + 0.7 * \log(\lambda_{m,t-1}) + \nu_{m,t}\big),$$

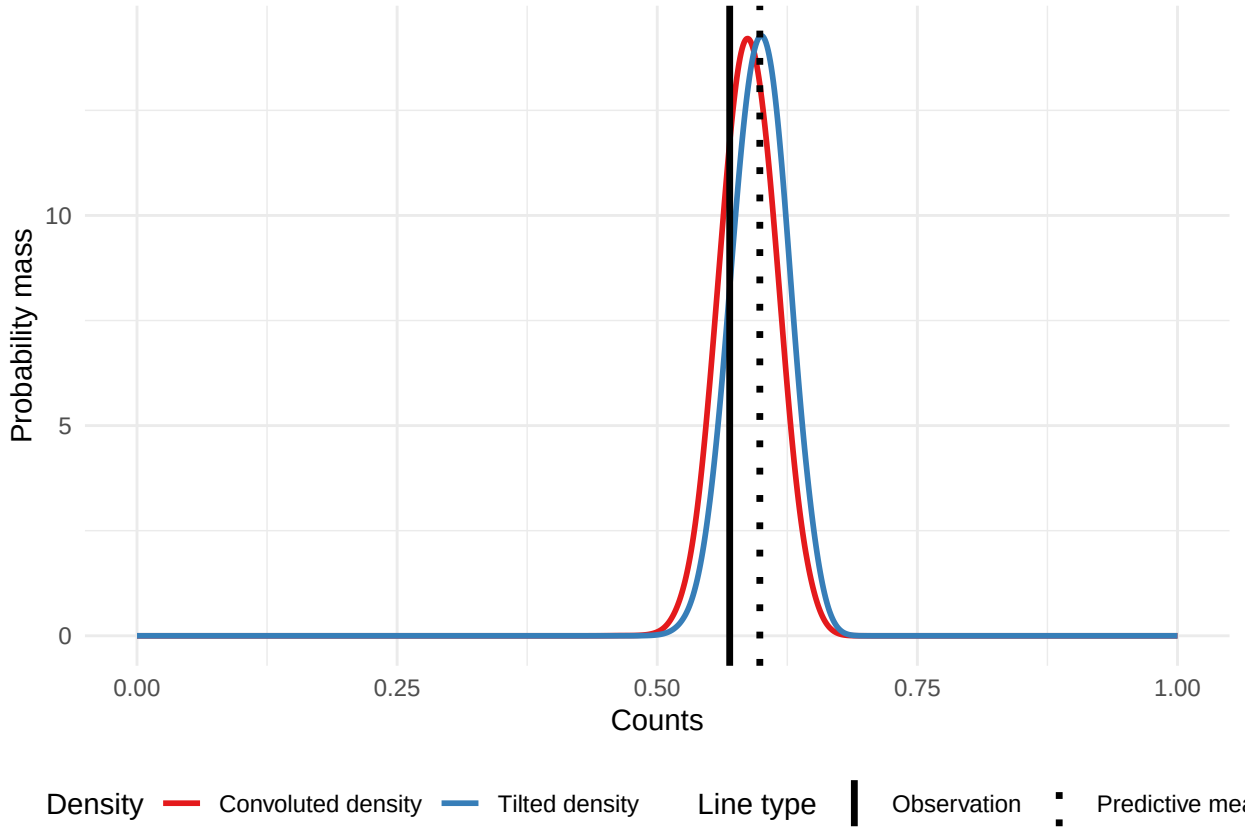

Figure 4: The tilted and convoluted density of the top series using Beta distributed bottom series nodes for a given time point. The dark red colour captures the density of the convolution, the blue represents the tilted denisty and the dashed black line indicates the predictive mean of the top series.

with observed counts generated according to

$$b_{m,t} \sim \text{Poisson}(\lambda_{m,t}).$$

To introduce additional variability, integer-valued shocks $\varepsilon_{m,t} \in \{-1, 0, 1\}$ are added independently to each $b_{m,t}$. The distribution of $\varepsilon_{i,m}$ is series-specific to ensure that noise does not systematically cancel out when aggregating across series:

$$\begin{aligned}
\varepsilon_{1,t} &\sim \{-1, 0, 1\} \text{ with probabilities } (0.4, 0.4, 0.2),\\
\varepsilon_{2,t} &\sim \{-1, 0, 1\} \text{ with probabilities } (0.2, 0.7, 0.1),\\
\varepsilon_{3,t} &\sim \{-1, 0, 1\} \text{ with probabilities } (0.33, 0.34, 0.33),\\
\varepsilon_{4,t} &\sim \{-1, 0, 1\} \text{ with probabilities } (0.2, 0.5, 0.3).
\end{aligned}$$

To ensuring non-negativity, the perturbed bottom-level series are then given by

$$b^*_{m,t} = \max\big(0,\, b_{m,t} + \varepsilon_{m,t}\big).$$

At the aggregate level, the series is obtained by summing across the unperturbed bottom-level counts:

$$u_t = \sum_{m=1}^{M} b_{m,t}.$$

The final noisy dataset consists of the covariate x, the aggregate series u, and the four perturbed bottom-level series $(b^*_1, \ldots, b^*_4)$.

Data generating process for Case 2: Hierarchical Beta distributions.

The bottom-level series $(b_{AA,t}, b_{AB,t})$ are constructed from autoregressive latent processes on the logit scale with correlated Gaussian noise terms. Specifically, for $m = \{AA, AB\}$, $t = 1, \ldots, N^\star$ where $N^\star = 400$ and burn-in $T_0 = 150$, we define

$$\text{logit}(b_{m,1}) \sim \text{Normal}(0, 1), \text{logit}(b_{m,t}) \quad = \beta_{0,m} + \beta_{1,m}\, b_{m,t-1} + \epsilon_{m,t}, \tag{6}$$

where, $b_{m,t}$ is the proportion of bottom-level series m at time t.

We set the coefficients of the data generating process to be,

$$\beta_{0,AA} = 0.85, \quad \beta_{1,AA} = 0.50, \beta_{0,AB} = 0.50, \quad \beta_{1,AB} = 0.40, \boldsymbol{\epsilon}_t \quad \sim \text{Normal}_2\big((00)\,, \Sigma\big)\,, \quad \Sigma = \big((0.15)^2 \quad -0.6(0.15)^2 - 0.6(0.15$$

The bottom-level proportions are obtained by the inverse-logit transform. We discard the first T_0 observations to ensure that the AR process is stable. We define the mean proportion for the higher series node $u_{A,t}$ as the weighted sum of the lower–level series $b_{AA,t}$ and $b_{AB,t}$, $\bar{u}_{A,t} = 0.75 b_{AA,t} + 0.25 b_{AB,t}$. To prevent a deterministic relationship between the nodes, we introduce noise via a concentration parameter $\phi_A = 500$. We set ϕ_A to be very small to prevent the signal being overwhelmed by noise induced by the sampling from the Beta distribution. Finally, we simulate draws from a Beta distribution,

$$u_{A,t} \sim \text{Beta}\big(\phi_A \bar{y}_{A,t},\ \phi_A(1 - \bar{u}_{A,t})\big). \tag{7}$$

where, $u_{A,t}$ is the observed proportion of top-level series A at time t.
The complementary proportion for category B is defined as $u_{B,t} = 1 - u_{A,t}$. The final simulated dataset consists of 250 observations of the hierarchical composition $\{u_{A,t}, u_{B,t}, b_{AA,t}, b_{AB,t}\}$.

Bibliography

G. Athanasopoulos, R. J. Hyndman, N. Kourentzes, and A. Panagiotelis. Forecast reconciliation: A review. *International Journal of Forecasting*, 40:430–456, 4 2024. ISSN 01692070. doi: 10.1016/j.ijforecast.2023.10.010. [p1]

D. Azzimonti, N. Rubattu, L. Zambon, and G. Corani. *bayesRecon: Probabilistic Reconciliation via Conditioning*, 2025. URL https://CRAN.R-project.org/package=bayesRecon. R package version 0.3.3. [p1]

R. L. Burden and J. D. Faires. *Numerical Analysis*. Cengage Learning, Boston, MA, 9th edition, 2015. ISBN 978-1-305-06499-6. [p2]

G. Corani, D. Azzimonti, and N. Rubattu. Probabilistic reconciliation of count time series. *International Journal of Forecasting*, 39(1):1–17, 2023. doi: 10.1016/j.ijforecast.2022.10.012. [p1]

R. Giacomini and G. Ragusa. Theory-coherent forecasting. *Journal of Econometrics*, 182(1):145–155, Sept. 2014. doi: 10.1016/j.jeconom.2014.04.003. [p2]

D. Girolimetto and T. Di Fonzo. *FoReco: Forecast Reconciliation*. R package version 1.0, 2024. URL https://cran.r-project.org/package=FoReco. [p1]

T. Gneiting and A. E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007. [p1]

GO FAIR Initiative. *FAIR Principles*, 2016. URL https://www.go-fair.org/fair-principles/. Accessed: 2026-03-18. [p8]

R. Hyndman, A. Lee, E. Wang, and S. Wickramasuriya. *hts: Hierarchical and Grouped Time Series*, 2024. URL https://CRAN.R-project.org/package=hts. R package version 6.0.3. [p1]

R. J. Hyndman. Notation for forecast reconciliation, 11 2022. URL https://robjhyndman.com/hyndsight/reconciliation-notation.html. Accessed: 2025-06-17. [p1]

R. J. Hyndman and G. Athanasopoulos. *Forecasting: principles and practice*. 2021. ISBN 978-0987507136. URL https://otexts.com/fpp3/. Accessed: 2025-06-17. [p1]

R. J. Hyndman, R. A. Ahmed, G. Athanasopoulos, and H. L. Shang. Optimal combination forecasts for hierarchical time series. *Computational Statistics and Data Analysis*, 55:2579–2589, 9 2011. ISSN 01679473. doi: 10.1016/j.csda.2011.03.006. [p1]

F. Krüger, T. E. Clark, and F. Ravazzolo. Using entropic tilting to combine bvar forecasts with external nowcasts. *Journal of Business & Economic Statistics*, 35(3):470–485, 2017. doi: 10.1080/07350015.2015.1087856. [p2]

M. O'Hara-Wild, R. Hyndman, and E. Wang. *fabletools: Core Tools for Packages in the 'fable' Framework*, 2025. URL https://CRAN.R-project.org/package=fabletools. R package version 0.5.1. [p1]

A. Panagiotelis. *ProbReco: Score Optimal Probabilistic Forecast Reconciliation*, 2025. URL https://github.com/anastasiospanagiotelis/ProbReco. R package version 0.1.2, commit fd4bacbd4cd793d5c37447fad02f16cb940c5182. [p1]

A. Panagiotelis, P. Gamakumara, G. Athanasopoulos, and R. J. Hyndman. Probabilistic forecast reconciliation: Properties, evaluation and score optimisation. *European Journal of Operational Research*, 306:693–706, 4 2023. ISSN 03772217. doi: 10.1016/j.ejor.2022.07.040. [p1]

S. B. Taieb, J. W. Taylor, and R. J. Hyndman. *Coherent Probabilistic Forecasts for Hierarchical Time Series*, 2017. [p1]

M. West. Perspectives on constrained forecasting. *Bayesian Analysis*, 19(4):1013–1039, 2024. doi: https://doi.org/10.1214/23-BA1379. [p2]

S. L. Wickramasuriya, G. Athanasopoulos, and R. J. Hyndman. Optimal forecast reconciliation for hierarchical and grouped time series through trace minimization. *Journal of the American Statistical Association*, 114:804–819, 4 2019. ISSN 1537274X. doi: 10.1080/01621459.2018.1448825. [p1]

Hannah Comiskey
Monash Univeristy
Department of Econcometrics and Business Statistics
Melbourne, Australia
https://hannahcomiskey.github.io/
ORCiD: 0000-0003-0034-6725
hannahcomiskey@monash.edu

David Frazier
Monash Univeristy
Department of Econcometrics and Business Statistics
Melbourne, Australia

Ole Maneesoonthorn
Monash Univeristy
Department of Econcometrics and Business Statistics
Melbourne, Australia